

# Structure-based techniques



# Introduction

- Structure-based testing/white-box testing is based on an identified structure of the software or system
- It is a method for writing a set of white-box test cases that exercise the paths in the code
- How to measure thoroughly the test cases exercise the code

# What is test coverage?

- Test coverage measures in some specific way the amount of testing performed by a set of tests.
- Wherever we can count things and can tell whether or not each of those things has been tested by some test, then we can measure coverage

$$\text{Coverage} = \frac{\text{Number of coverage items exercised}}{\text{Total number of coverage items}} \times 100\%$$

# Basic white box test case design techniques

- Statement testing and coverage
- Decision testing and coverage
- Other structure-based techniques



# Definition of statement coverage

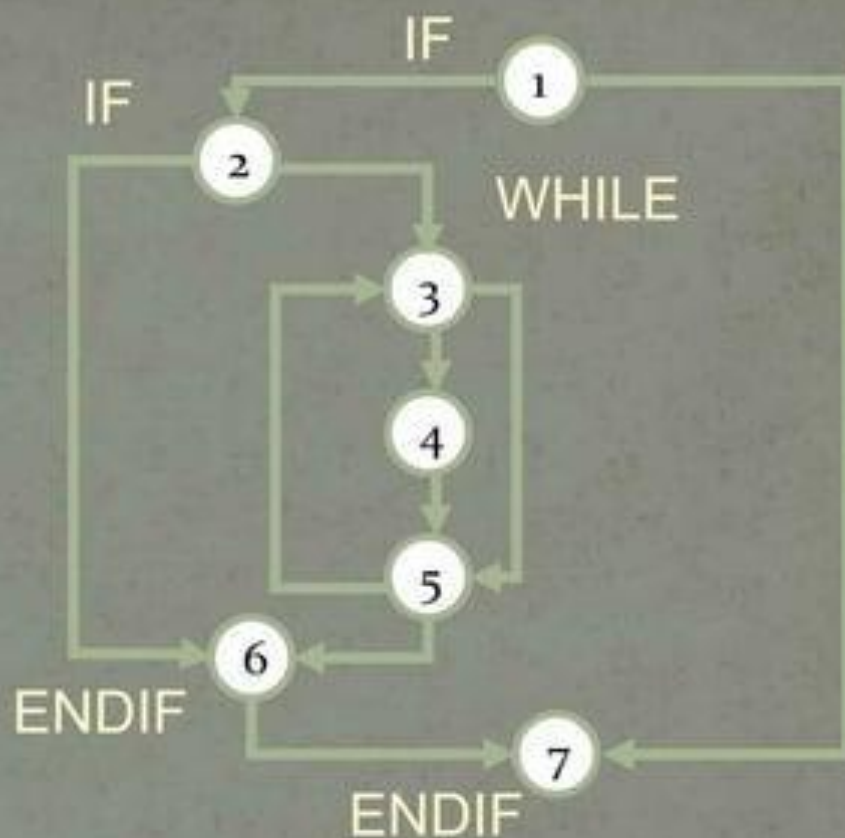
- In component testing, statement coverage is the assessment of the percentage of executable statements that have been exercised by a test case suite. (TRUE OR FALSE)
- Statement testing derives test cases to execute specific statements, normally to increase statement coverage.
- Every statement is requested to be executed at least one

# Steps

- Translate the source code to control flow graph
- Defines nodes(Statement) and control flow between statements(edges)
  - Conditional statements(IF, CASE) and loops(WHILE, FOR) have more than one edges going out
- Verify that each statement have been executed

# Examples

```
1 If (a>0) then
2   If (b<1) then
3     While
4       (c<2)
5       C++;
6   End IF
7 End IF
```

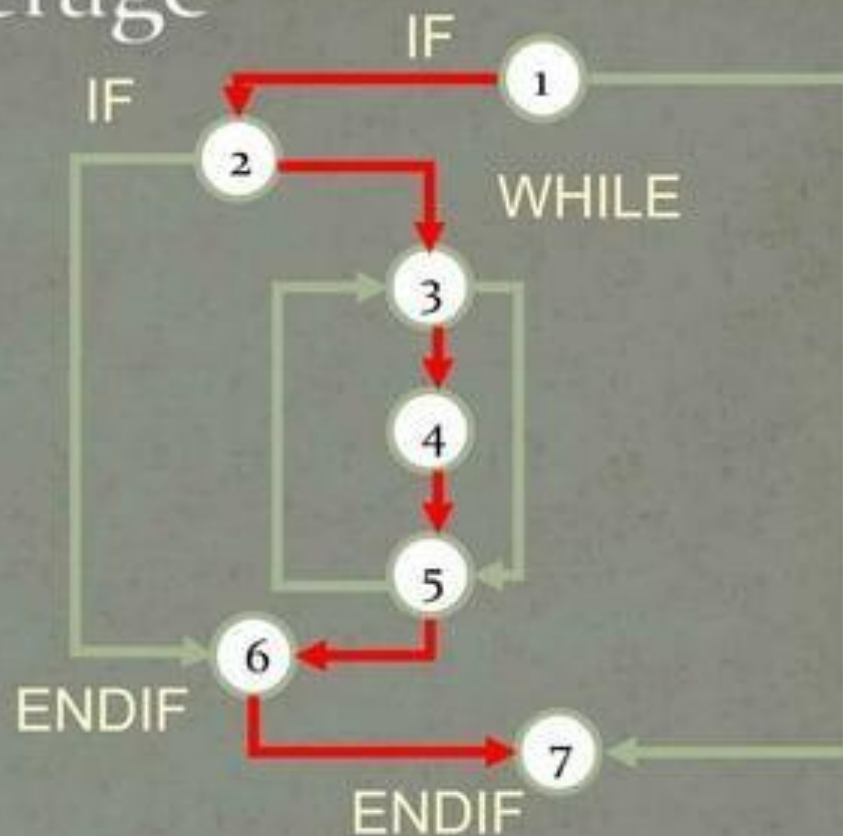


○ Node, statement

→ Edge, Control flow

# Statement coverage

- 100%SC: All nodes can be reached by a single test case



1, 2, 3, 4, 5, 6, 7

$$\text{Statement coverage} = \frac{\text{Number of statements exercised}}{\text{Total number of statements}} \times 100\%$$



# Statement Coverage

- It is a very weak criterion
- Value of the technique:
  - Unreachable source code (**dead code**) can be detected
  - Empty ELSE-parts are not considered

➡ Coverage is measured by using test tools

# Exercise

- What is minimum number of test cases required for full statement coverage?
  - a) 1 test for statement coverage
  - b) 2 tests for statement coverage
  - c) 3 tests for statement coverage

*Read P*

*Read Q*

*IF  $P+Q > 100$  THEN*

*Print "Large"*

*ENDIF*

*If  $P > 50$  THEN*

*Print "P Large"*

*ENDIF*

# Definition of decision/branch coverage

- Decision coverage, related to branch coverage, is the assessment of the percentage of decision outcomes (TRUE and FALSE)
- Require every decision outcome: all possible cases-statement
- Decision testing is a form of control flow testing as it generates a specific flow of control through the decision points

# Steps

- Translate the source code to control flow graph
- Defines nodes(Statement) and control flow between statements(edges)
  - Conditional statements(IF, CASE) and loops(WHILE, FOR) have more than one edges going out
- Verify that all possible branches of the control flow are tested

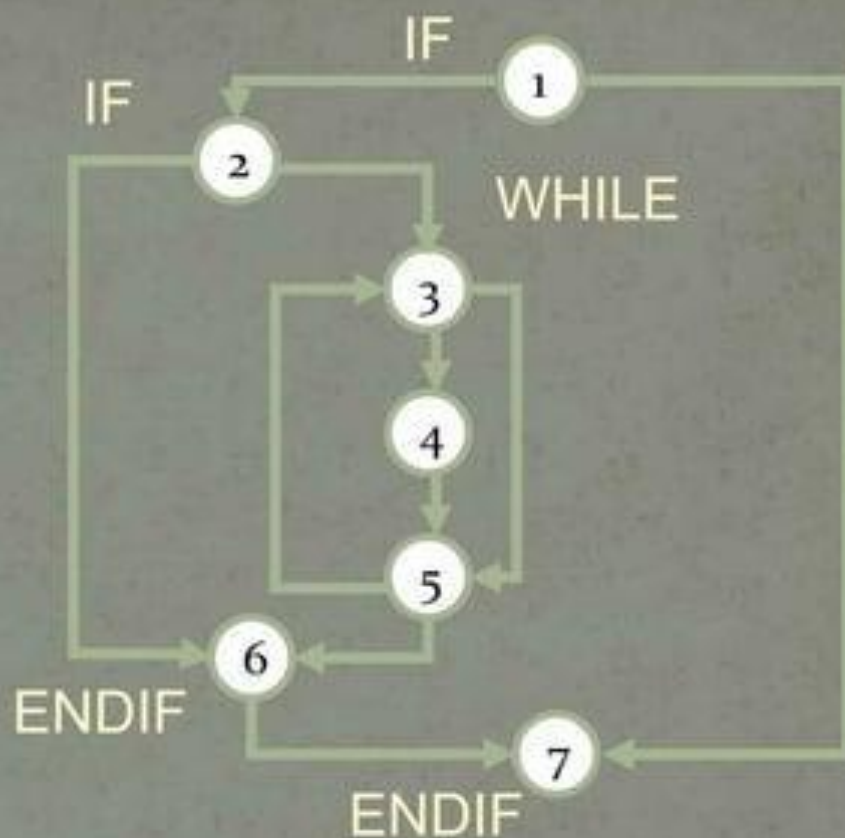


# Decision/branch Coverage

- It is a stronger criterion than statement coverage
- Be able to detect missing statement in empty branches
- Decision coverage is stronger than statement coverage: **100% decision coverage guarantees 100% statement coverage, but not vice versa.**
- Decision coverage requires test of every decision outcome:
  - both THEN and ELSE in the IF-statement
  - all possibilities for the CASE-statement and fall-through case
  - both execution of the FOR loop and the bypass

# Examples

```
1 If (a>0) then
2   If (b<1) then
3     While
4       (c<2)
5       c++;
6     End while
7   End IF
8 End IF
```



- Node, statement
- Edge, Control flow

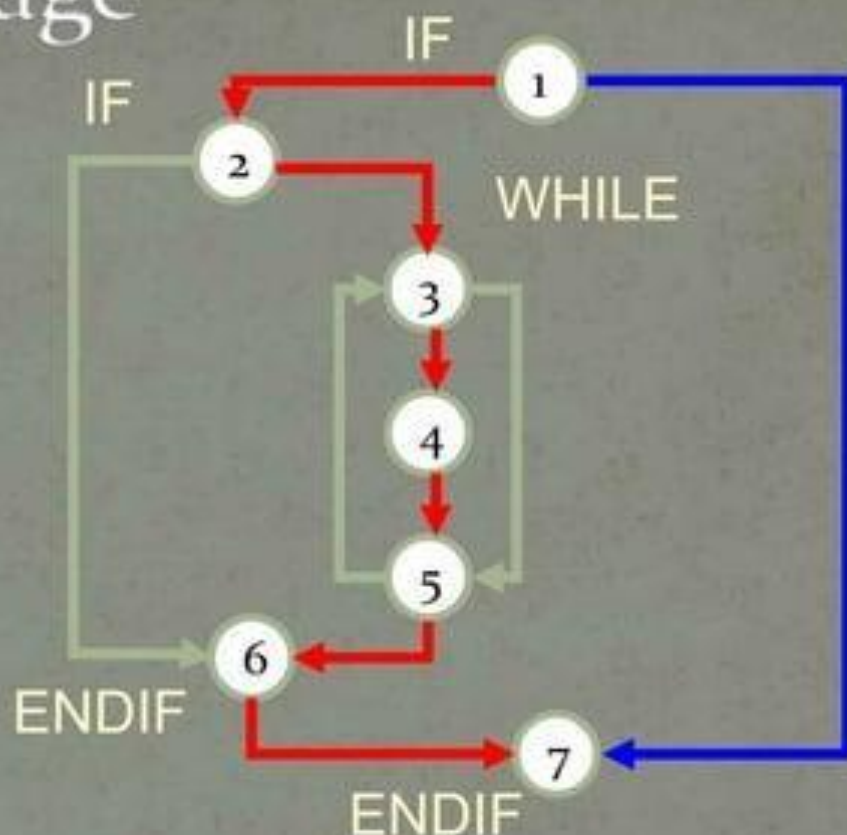
# Decision coverage

- 100%DC: All branches are executed
- Some edges are executed more than one

1, 2, 3, 5, 3, 4, 5, 6, 7

1, 2, 6, 7

1, 7



- Branch coverage = (Number of executed branches / total number of branches) \* 100%



# Exercise

- What is minimum number of test cases required for full decision coverage?
  - a) 1 test for decision coverage
  - b) 2 tests for decision coverage
  - c) 3 tests for decision coverage

*Switch PC on*

*Start "outlook"*

*IF outlook appears THEN*

*Send an email*

*Close outlook*

*Endif*



# Limitations of the techniques

- For object-oriented system, statement as well as decision coverage is inadequate.
- Additional coverage criteria are necessary
- Tools often support determining coverage, coverage data can be used to detect uncalled methods or program parts

# Other structure-based test techniques

- Data flow Testing
- Branch Condition Testing
- Branch Condition Combination Testing
- Modified Condition Decision Testing
- LCSAJ Testing

# Other structure-based test techniques (cont.)

- There are stronger levels of structural coverage beyond decision coverage (condition coverage and multiple condition coverage)
- The concept of coverage can also be applied at other test levels (e.g. at integration level) where the percentage of modules, components or classes that have been exercised by a test case suite could be expressed as module, component or class coverage
- Tool support is useful for the structural testing of code
- The basis of white box techniques is the source code. Adequate test case design techniques chosen depending on the complexity of the program structure

# Which techniques should be chosen?

- Testing contains the combination of different techniques
- The expected risk guides to select testing techniques and the intensity of execution
- Basis of the selection of the white box technique is the structure of the test object



# Cyclomatic Complexity



# Definition

- Cyclomatic complexity or conditional complexity:
  - Was developed by McCabe
  - Is the software metric (measurement) to measure the complexity of the program's source code

# How to calculate

- $M = E - N + 2P$ 
  - $M$  = cyclomatic complexity
  - $E$  = the number of edges (links) of the graph
  - $N$  = the number of nodes of the graph
  - $P$  = the number of connected components
- **Number of predicates + 1**

# Examples of cyclomatic complexity



$$E=1, N=2, P=1$$
$$M=1-2+2=1$$



$$E=4, N=4, P=1$$
$$M=4-4+2=2$$



$$E=2, N=4, P=2$$
$$M=2-4+4=2$$



$$E=4, N=5, P=1$$
$$M=4-5+2=1$$



# Cyclomatic complexity of programming constructs

```
1 if E then  
2   A  
   else  
3   B  
4 C
```

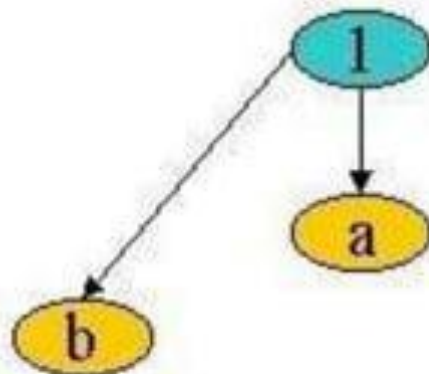


$$E = 4, N = 4, P = 1$$
$$M = 2$$

```

void Test::Test1()
{
    string a,b,c,d,e,f;
1   if (a == b)
a       c = d;
    else
b       c = f;
}

```



Predicate Nodes  $P = 1$   
 $= 1 + 1 = 2$